Tekirdağ Namik Kemal Üniversitesi

Bilgisayar Mühendisliği Bölümü

Uzaysal Filtreleme Yöntemleri ile Görüntü Gürültü Azaltma

Hazırlayan

Besat Çıngar

11 Mart 2026

Özet

Görüntüler gerçek dünyadan elde edilirken sensör hataları, iletim problemleri ve çevresel etkiler nedeniyle çeşitli gürültüler içerebilir. Bu gürültüler görüntü kalitesini düşürmekte ve görüntü işleme algoritmalarının performansını olumsuz etkilemektedir.

Bu çalışmada ortalama filtre, Gaussian filtre ve medyan filtre gibi uzaysal filtreleme yöntemleri, hazır kütüphane fonksiyonlarına bağlı kalınmadan Python kullanılarak algoritmik olarak gerçeklenmiştir. Üç farklı gürültülü görüntü üzerinde deneyler yapılarak filtrelerin performansları karşılaştırılmıştır.

1 Giriş

Sayısal görüntü işleme, görüntülerin bilgisayar ortamında analiz edilmesi ve geliştirilmesini sağlayan teknikleri içermektedir. Gerçek dünyadan elde edilen görüntüler çoğu zaman çeşitli gürültüler barındırmaktadır.

Gürültü, görüntü kalitesini düşürmekte ve görüntüdeki önemli detayların kaybolmasına neden olabilmektedir. Bu nedenle görüntü işleme süreçlerinde gürültü azaltma (noise reduction) oldukça önemli bir ön işleme adımıdır.

Uzaysal filtreleme yöntemleri, bir pikselin yeni değerini, o pikselin kendisi ve komşu piksellerinin değerleri ile ilişkilendirerek hesaplamaya dayanmaktadır.

2 Uzaysal Filtreleme

Uzaysal filtreleme işlemi bir çekirdek (kernel) veya maske kullanılarak gerçekleştirilir. Bu maske görüntü matrisi üzerinde kaydırılarak her piksel için konvolüsyon işlemi uygulanır ve yeni değer hesaplanır.

İki boyutlu ayrık konvolüsyon işlemi aşağıdaki şekilde ifade edilmektedir:

$$g(x, y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} f(x - i, y - j)h(i, j)$$

Burada;

- $f(x, y)$: Giriş görüntüsünü,
- $h(i, j)$: Filtre çekirdeğini (maskeyi),
- $g(x, y)$: Çıktı görüntüsünü ifade etmektedir.

Ayrıca filtre maskesinin boyutunun $(2k + 1) \times (2k + 1)$ olduğu varsayılmaktadır.

3 Ortalama Filtre

Ortalama (Mean) filtre, bir pikselin değerini, maske boyutunca belirlenen komşu piksellerin aritmetik ortalaması ile değiştirir.

Standart bir 3x3 ortalama filtre çekirdeği şu şekildedir:

$$H = \frac{1}{9}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

Bu filtre genel gürültüyü azaltmada etkilidir ancak yüksek frekanslı bileşenleri de (kenarlar ve keskin detaylar) zayıflattığı için görüntüde bulanıklaşmaya neden olabilmektedir.

4 Gaussian Filtre

Gaussian filtre, ortalama filtreye benzer şekilde çalışır ancak piksellerin ağırlıkları, merkezden uzaklaştıkça azalacak şekilde iki boyutlu Gaussian dağılımına göre belirlenir. Dağılım fonksiyonu şu şekildedir:

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

Bu çalışmada yaklaşıklık kullanılarak elde edilmiş 3x3 boyutundaki standart Gaussian çekirdeği kullanılmıştır:

$$G = \frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

5 Medyan Filtre

Medyan filtre, doğrusal olmayan (non-linear) bir filtreleme yöntemidir. Bu yöntemde, maske altında kalan piksellerin değerleri küçükten büyüğe sıralanır ve merkez pikselin değeri, bu dizinin ortancası (medyanı) ile değiştirilir.

Matematiksel olarak şu şekilde ifade edilir:

$$g(x, y) = \text{medyan}\{f(x + i, y + j)\}$$

Bu filtre, görüntüdeki kenar detaylarını koruma konusunda ortalama filtreye göre daha başarılıdır ve özellikle "tuz-biber" (salt-and-pepper) gürültüsünü gidermede oldukça etkilidir.

6 Uygulama

Filtreleme işlemleri Python programlama dili kullanılarak, temel matematiksel işlemler koda dökülerek gerçekleştirilmiştir. OpenCV ve NumPy kütüphaneleri yalnızca görüntülerin okunması, veri matrislerinin oluşturulması ve sonuçların ekrana yazdırılması amacıyla kullanılmıştır. Uygulanan Python betiği, belgenin sonundaki Ek bölümünde verilmiştir.

Programın temel işleyişi şu şekildedir:

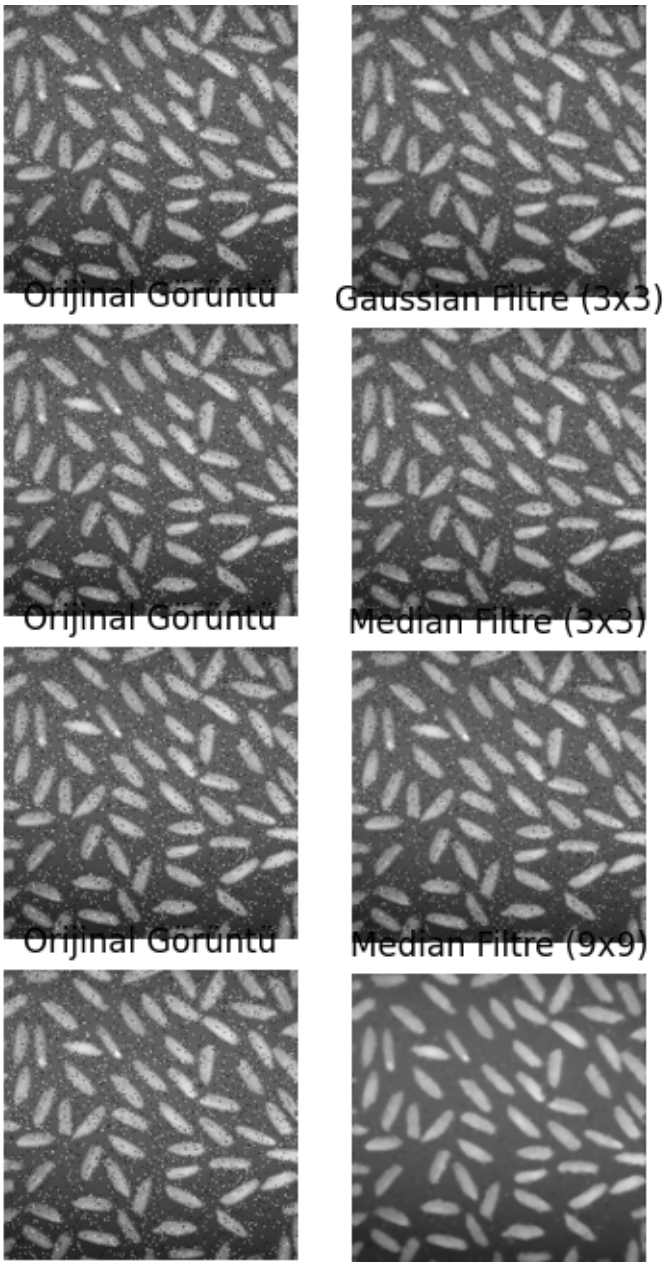
- Görüntünün gri tonlamalı (grayscale) olarak okunması,
- Ortalama filtrenin pikseller üzerinde gezilerek uygulanması,
- Gaussian filtrenin uygulanması,

- Medyan filtrenin uygulanması,
- Elde edilen sonuçların görselleştirilerek karşılaştırılması.

7 Deneysel Sonuçlar

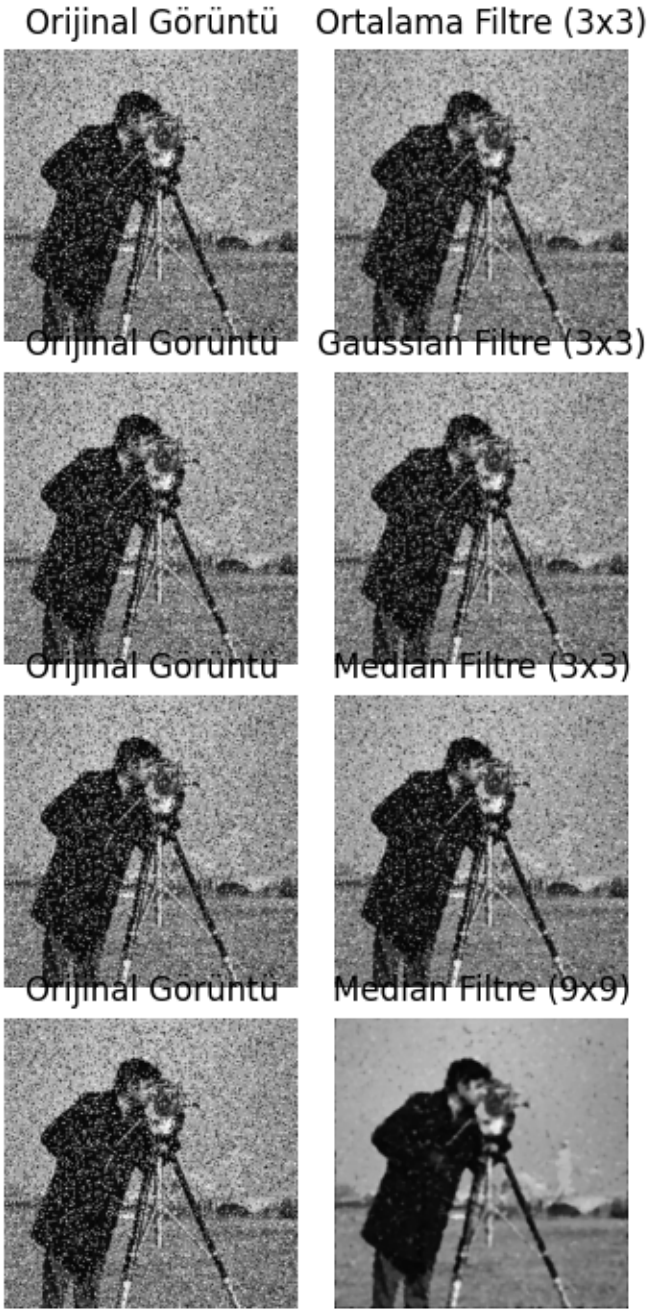
Bu çalışmada, yapay olarak gürültü eklenmiş üç farklı görüntü üzerinde yukarıda bahsedilen filtreleme işlemleri uygulanmıştır.

7.1 Görüntü 1 Sonuçları



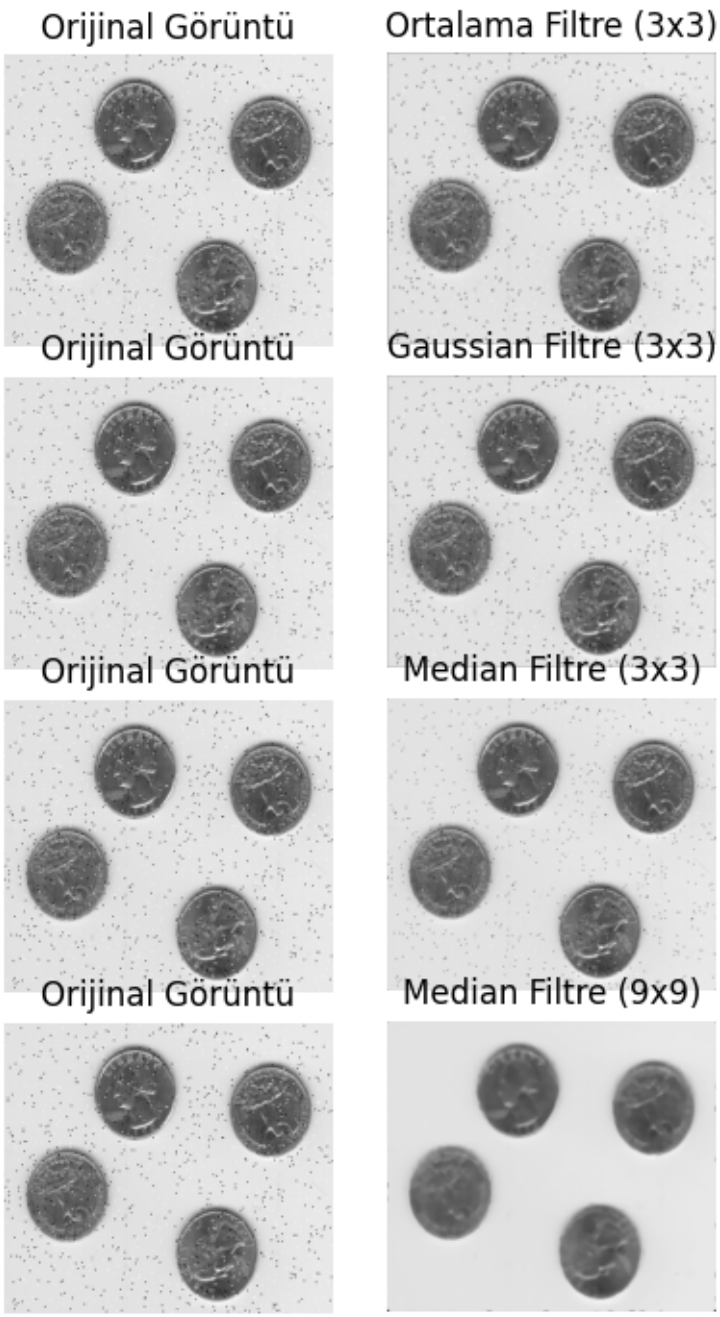
Şekil 1: 1 numaralı görüntü için filtreleme sonuçları

7.2 Görüntü 2 Sonuçları



Şekil 2: 2 numaralı görüntü için filtreleme sonuçları

7.3 Görüntü 3 Sonuçları



Şekil 3: 3 numaralı görüntü için filtreleme sonuçları

8 Sonuç

Bu çalışmada ortalama, Gaussian ve medyan filtreleme yöntemleri teorik olarak incelenmiş ve Python ortamında algoritmik olarak uygulanmıştır.

Deneysel sonuçlar, medyan filtrenin özellikle impuls (tuz-biber) gürültüsünü gidermede oldukça başarılı olduğunu ve nesne sınırlarını koruduğunu göstermektedir. Gaussian filtre ise kenarları ortalama filtreye kıyasla daha iyi koruyarak daha doğal ve dengeli bir yumuşatma sağlamaktadır.

Gelecek çalışmalarda, bilateral filtreleme gibi kenar koruyucu daha gelişmiş yöntemler denenebilir ve filtre performansları PSNR (Tepe Sinyal Gürültü Oranı) gibi sayısal ölçütler kullanılarak daha nesnel bir şekilde analiz edilebilir.

A Python Kaynak Kodları

```python
import cv2
import numpy as np
import matplotlib.pyplot as plt
import os

def apply_linear_filter(image, kernel):
    k_h, k_w = kernel.shape
    pad_h, pad_w = k_h // 2, k_w // 2

    padded_image = np.pad(image, ((pad_h, pad_h), (pad_w, pad_w)), mode='constant', constant_values=0)
    output_image = np.zeros_like(image, dtype=np.float64)

    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            region = padded_image[i:i+k_h, j:j+k_w]
            output_image[i, j] = np.sum(region * kernel)

    return np.clip(output_image, 0, 255).astype(np.uint8)

def apply_nonlinear_filter(image, kernel_size, operator):
    pad = kernel_size // 2
    padded_image = np.pad(image, ((pad, pad), (pad, pad)), mode='constant', constant_values=0)
    output_image = np.zeros_like(image)

    for i in range(image.shape[0]):
        for j in range(image.shape[1]):
            region = padded_image[i:i+kernel_size, j:j+kernel_size]

            if operator == 'min':
                output_image[i, j] = np.min(region)
            elif operator == 'max':
                output_image[i, j] = np.max(region)
            elif operator == 'median':
                output_image[i, j] = np.median(region)

    return output_image

image_list = [
    'kaynak_resmi.png',
    'ortalama_filtre.png',
    'gurultulu_resim_gaussian.png',
    'median.png',
    'median_2.png'
]

for filename in image_list:
    if not os.path.exists(filename):
        continue

    img = cv2.imread(filename, cv2.IMREAD_GRAYSCALE)

    kernel_avg = np.ones((3, 3), np.float32) / 9.0
    avg_filtered = apply_linear_filter(img, kernel_avg)

```

```
    kernel_gaussian = np.array([[1, 2, 1],
                                [2, 4, 2],
                                [1, 2, 1]], dtype=np.float32) / 16.0
    gaussian_filtered = apply_linear_filter(img, kernel_gaussian)

    median_filtered_3x3 = apply_nonlinear_filter(img, 3, 'median')
    median_filtered_9x9 = apply_nonlinear_filter(img, 9, 'median')

    name, ext = os.path.splitext(filename)
    cv2.imwrite(f"{name}_sonuc_ortalama{ext}", avg_filtered)
    cv2.imwrite(f"{name}_sonuc_gaussian{ext}", gaussian_filtered)
    cv2.imwrite(f"{name}_sonuc_median3x3{ext}", median_filtered_3x3)
    cv2.imwrite(f"{name}_sonuc_median9x9{ext}", median_filtered_9x9)

    fig, axes = plt.subplots(nrows=4, ncols=2, figsize=(10, 16))
    fig.suptitle(f"Goruntu: {filename}", fontsize=16, fontweight='bold')

    axes[0, 0].imshow(img, cmap='gray', vmin=0, vmax=255)
    axes[0, 0].set_title("Orijinal Goruntu")
    axes[0, 0].axis('off')
    axes[0, 1].imshow(avg_filtered, cmap='gray', vmin=0, vmax=255)
    axes[0, 1].set_title("Ortalama Filtre (3x3)")
    axes[0, 1].axis('off')

    axes[1, 0].imshow(img, cmap='gray', vmin=0, vmax=255)
    axes[1, 0].set_title("Orijinal Goruntu")
    axes[1, 0].axis('off')
    axes[1, 1].imshow(gaussian_filtered, cmap='gray', vmin=0, vmax=255)
    axes[1, 1].set_title("Gaussian Filtre (3x3)")
    axes[1, 1].axis('off')

    axes[2, 0].imshow(img, cmap='gray', vmin=0, vmax=255)
    axes[2, 0].set_title("Orijinal Goruntu")
    axes[2, 0].axis('off')
    axes[2, 1].imshow(median_filtered_3x3, cmap='gray', vmin=0, vmax
=255)
    axes[2, 1].set_title("Median Filtre (3x3)")
    axes[2, 1].axis('off')

    axes[3, 0].imshow(img, cmap='gray', vmin=0, vmax=255)
    axes[3, 0].set_title("Orijinal Goruntu")
    axes[3, 0].axis('off')
    axes[3, 1].imshow(median_filtered_9x9, cmap='gray', vmin=0, vmax
=255)
    axes[3, 1].set_title("Median Filtre (9x9)")
    axes[3, 1].axis('off')

    plt.tight_layout()
    plt.subplots_adjust(top=0.95)
    plt.show()
```

Listing 1: Görüntü Filtreleme Uygulaması Kaynak Kodu